

Given a code example, determine if a method is correctly overriding or overloading another method, and identify legal return values (including covariant returns), for the method.

Covariant return types

You cannot have two methods in the same class with signatures that only differ by return type. Until the J2SE 5.0 release, it was also true that a class could not override the return type of the methods it inherits from a superclass. J2SE 5.0 allows covariant return types. What this means is that a method in a subclass may return an object whose type is a subclass of the type returned by the method with the same signature in the superclass. This feature removes the need for excessive type checking and casting.

Let's start with the following class, `ConfusedClass`. The class tries to declare two methods with the same signature. One of the methods returns a `TextField`, and the other returns a `PasswordField`.

```
import javax.swing.JTextField;
import javax.swing.PasswordField;

public class ConfusedClass {

    public JTextField getTextField(){
        return new JTextField();
    }

    public PasswordField getTextField(){
        return new PasswordField();
    }

}
```

If you try to compile `ConfusedClass`, you get the following compile error:

```
ConfusedClass.java:10: getTextField() is already defined in ConfusedClass
    public PasswordField getTextField(){
        ^
    1 error
```

Looking at this situation from the perspective of a class calling `getTextField()`, you can see the reason for the compile time error. How would you indicate which of the two methods you are targeting? Consider, for example, this snippet:

```
ConfusedClass cc = new ConfusedClass();
JTextField field = cc.getTextField();
```

Because a `PasswordField` extends `JTextField`, either version of the method could correctly be called.

Next, create two classes, each of which having a different version of the `getTextField()` methods. The two methods differ by implementation and return type. Start with the following base:

```
import javax.swing.JTextField;

public class ConfusedSuperClass {
```

```

        public JTextField getTextField(){
            System.out.println("Called in " + this.getClass());
            return new JTextField();
        }
    }
}

```

Compile ConfusedSuperClass. You'll see that it compiles without error. Now create a derived class, one that extends ConfusedSuperClass. The derived class attempts to return an instance of JPasswordField instead of the JTextField returned by the getTextField() method in ConfusedSuperClass.

```

import javax.swing.JPasswordField;

public class ConfusedSubClass extends ConfusedSuperClass {
    public JPasswordField getTextField(){
        System.out.println("Called in " + this.getClass());
        return new JPasswordField();
    }
}

```

If you use a version of the JDK prior to J2SE 5.0, ConfusedSubClass will not compile. You will see an error like this:

```

ConfusedSubClass.java:5: getTextField() in ConfusedSubClass
cannot override getTextField() in ConfusedSuperClass;
attempting to use incompatible return type
found   : javax.swing.JPasswordField
required: javax.swing.JTextField
    public JPasswordField getTextField(){
           ^
    1 error

```

The error reported is that you are attempting to use an incompatible return type. In fact, the JPasswordField you are attempting to return is a subtype of JTextField. This same code compiles correctly under J2SE 5.0. You are now allowed to override the return type of a method with a subtype of the original type. In the current example, the getTextField() method in ConfusedSuperClass returns an instance of type JTextField. The getTextField() method in the ConfusedSubClass returns an instance of type JPasswordField.

You can exercise these two classes with the following NotConfusedClient class. This class creates an instance of type ConfusedSuperClass and of type ConfusedSubClass. It then calls getTextField() on each instance, and displays the type corresponding to the object returned by the method:

```

import javax.swing.JTextField;

public class NotConfusedClient {
    static JTextField jTextField;

    public static void main(String[] args) {
        System.out.println("==== Super Class =====");
        jTextField = new ConfusedSuperClass().getTextField();
        System.out.println("Got back an instance of " + jTextField.getClass());

        System.out.println("==== Sub Class =====");
        jTextField = new ConfusedSubClass().getTextField();
        System.out.println("Got back an instance of " + jTextField.getClass());
    }
}

```

Compile and run NotConfusedClient. When you run it, you should see the following output:

```
===== Super Class =====  
Called in class ConfusedSuperClass  
Got back an instance of class javax.swing.JTextField  
===== Sub Class =====  
Called in class ConfusedSubClass  
Got back an instance of class javax.swing.JPasswordField
```

In fact, you will get the same output if you change the return type of `getTextField()` to `JTextField` in `ConfusedSubClass`. The payoff comes when you use the object that is returned by the call to `getTextField()`. Before J2SE 5.0, you needed to downcast to take advantage of methods that are present in the derived class but not in the base class. As you've seen, in J2SE 5.0 `ConfusedSubClass` compiles with a different return type specified for `getTextField()` than is present in the superclass. You can now use your covariant return type to call a method that is only available in the subtype. First, recast the supertype like this:

```
public class SuperClass {  
    public SuperClass getAnObject(){  
        return this;  
    }  
}
```

Add the exclusive method to the corresponding subclass, and change the return type of the `getAnObject()` method:

```
public class SubClass extends SuperClass {  
    public SubClass getAnObject(){  
        return this;  
    }  
    public void exclusiveMethod(){  
        System.out.println("Exclusive in Subclass.");  
    }  
    public static void main(String[] args) {  
        System.out.println("===== Call Exclusive method =====");  
        new SubClass().getAnObject().exclusiveMethod();  
    }  
}
```

Compile `SuperClass` and `SubClass`, then run `SubClass`. You should see the following output:

```
===== Call Exclusive method =====  
Exclusive in Subclass.
```

The `main()` method creates an instance of the subclass. It then calls `getAnObject()`. It follows this with a call to `exclusiveMethod()` on the returned object of type `SubClass`. If the return type of `getAnObject()` was `SuperClass` in `SubClass.java`, the code would not have compiled.

Example of correct covariant return types:

```
Object <-- Number <-- Integer
```

```

import static java.lang.System.out;

class A {
    Object f() {
        return new Object();
    }
}

class B extends A {
    Number f() { // OK ! A.f() return (Object) is a superclass of B.f() return (Number)
        return Double.valueOf(1.0d);
    }
}

class C extends B {
    Integer f() { // OK ! B.f() return (Number) is a superclass of C.f() return (Integer)
        return 2;
    }
}

public class Client {
    public static void main(String ... sss) {
        A a = new A();
        A b = new B();
        A c = new C();

        out.println("a.f() instanceof Object : " + (a.f() instanceof Object));
        out.println("a.f() instanceof Number : " + (a.f() instanceof Number));
        out.println("b.f() instanceof Number : " + (b.f() instanceof Number));
        out.println("b.f() instanceof Integer : " + (b.f() instanceof Integer));
        out.println("c.f() instanceof Integer : " + (c.f() instanceof Integer));
    }
}

```

The output:

```

a.f() instanceof Object : true
a.f() instanceof Number : false
b.f() instanceof Number : true
b.f() instanceof Integer : false
c.f() instanceof Integer : true

```

Example of wrong covariant return types:

```

+-- String
Object <---|
+-- Numeric <-- Integer

'Integer' is NOT a subclass of 'String'

```

```

class A {
    Object f() {
        return new Object();
    }
}

class B extends A {
    String f() {
        return "sss";
    }
}

```

```
}  
}  
  
class C extends B {  
    Integer f() { // WRONG ! Compilation error ! The return type is incompatible with B.f()  
        return 2;  
    }  
}
```

[Prev](#)

Develop code that declares both static and non-static methods, and - if appropriate - use method names that adhere to the JavaBeans naming standards. Also develop code that declares and uses a variable-length argument list.

[Up](#)

[Home](#)

[Next](#)

Given a set of classes and superclasses, develop constructors for one or more of the classes. Given a class declaration, determine if a default constructor will be created, and if so, determine the behavior of that constructor. Given a nested or non-nested class listing, write code to instantiate the class.



SCDJWS 1.4
Quiz



ICAD
Study Guide



Magnet
Mocker

SCWCD 1.4
Study Guide

SCBCD 1.3
Study Guide

SCDJWS 1.4
Study Guide

IBM Test 287
Study Guide